ORF409 - PS2

Chapter - 3

(Q.2) $x_0 = 3$ ~~$x_1 = (5x_0+7) \% 200$~~

$x_1 = (5x_0+7) \% 200 = 22$

$x_2 = (5x_1+7) \% 200 = 117$

$x_3 = (5x_2+7) \% 200 = 192$

$x_4 = (5x_3+7) \% 200 = 967 \% 200 = 167$

$x_5 = (5x_4+7) \% 200 = 842 \% 200 = 42$

$x_6 = (5x_5+7) \% 200 = 217 \% 200 = 17$

$x_7 = (5x_6+7) \% 200 = (5 \times 17+7) \% 200 = (85+7) \% 200 = 92 \% 200 = 92$

$x_8 = (5 \times 92+7) \% 200 = 467 \% 200 = 67$

$x_9 = (5 \times 67+7) \% 200 = 342 \% 200 = 142$

$x_{10} = (5 \times 142+7) \% 200 = 717 \% 200 = 117$

ORF409 - PS2

Chapter - 3

(Q.2) $x_0 = 3$ ~~$+ x_1 = (5x_0+7) \% 200$~~

$x_1 = (5x_0+7) \% 200 = 22$

$x_2 = (5x_1+7) \% 200 = 117$

$x_3 = (5x_2+7) \% 200 = 192$

$x_4 = (5x_3+7) \% 200 = 967 \% 200 = 167$

$x_5 = (5x_4+7) \% 200 = 842 \% 200 = 42$

$x_6 = (5x_5+7) \% 200 = 217 \% 200 = 17$

$x_7 = (5x_6+7) \% 200 = (5 \times 17+7) \% 200 = (85+7) \% 200 = 92 \% 200 = 92$

$x_8 = (5 \times 92+7) \% 200 = 467 \% 200 = 67$

$x_9 = (5 \times 67+7) \% 200 = 342 \% 200 = 142$

$x_{10} = (5 \times 142+7) \% 200 = 717 \% 200 = 117$

Exact Answers:-

(Q.6) $\displaystyle\int_0^\infty \frac{x\,dx}{(1+x^2)^2}$

$\rightarrow t = x^2$

$\Rightarrow dt = 2x\,dx$

$\Rightarrow x\,dx = \frac{dt}{2}$

$x=0 \rightarrow t=0$

$x\to\infty \rightarrow t\to\infty$

$$= \int_0^\infty \frac{dt}{2(1+t)^2}$$

$$= \left[\frac{1}{2(1+t)}\right]_\infty^0$$

$$= \frac{1}{2}\left(\frac{1}{1}-\frac{1}{\infty}\right)$$

$$= \boxed{\frac{1}{2}}$$

(Q.7) we know that:-

$$\int_{-\infty}^{\infty} N(0,1)\,dx = \int_{-\infty}^{\infty} \frac{1}{\sqrt{2\pi}} e^{-x^2/2}\,dx = 1$$

$$\Rightarrow \int_{-\infty}^{\infty} e^{-x^2/2}\,dx = \sqrt{2\pi}$$

So, $\displaystyle\int_{-\infty}^{\infty} N(0,\tfrac{1}{2})\,dx = 1$

$$\Rightarrow \int_{-\infty}^{\infty} \frac{1}{\sqrt{\frac{1}{2}}\sqrt{2\pi}} e^{-\frac{x^2}{2\cdot\frac{1}{2}}}\,dx = 1$$

$$\Rightarrow \int_{-\infty}^{\infty} \frac{1}{\sqrt{\pi}} e^{-x^2}\,dx = 1$$

$$\Rightarrow \boxed{\int_{-\infty}^{\infty} e^{-x^2}\,dx = \sqrt{\pi}}$$

(Q.9) $\displaystyle\int_0^\infty\int_0^x e^{-x}e^{-y}\,dy\,dx$

$$= \int_0^\infty e^{-x}\left(\int_0^x e^{-y}\,dy\right)dx$$

$$= \int_0^\infty e^{-x}\left([e^{-y}]_x^0\right)dx$$

$$= \int_0^\infty e^{-x}(1-e^{-x})\,dx$$

$$= \int_0^\infty (e^{-x}-e^{-2x})\,dx$$

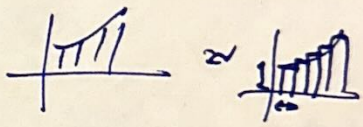
$$= [e^{-x}]_\infty^0 - \left[\frac{e^{-2x}}{2}\right]_\infty^0$$

$$= 1-0-\frac{1-0}{2}$$

$$= 1-\frac{1}{2}$$

$$= \boxed{\frac{1}{2}}$$

Simulation =



For (Q.6)

$\boxed{I = \int_0^\infty g(x)\,dx}$ $\left(g(x) = \frac{x}{(1+x^2)^2}\right)$

$\Rightarrow$ we want to bring $0-\infty$ to $0-1$

$\Rightarrow$ let $y = \frac{1}{x+1}$ $\Rightarrow dy = -\frac{1}{(x+1)^2}dx = -y^2\,dx$

$x\to 0 \Rightarrow y\to 1$, $x\to\infty \Rightarrow y\to 0$

$dx = -\frac{dy}{y^2}$

$$\Rightarrow I = -\int_1^0 \frac{g(\frac{1}{y}-1)}{y^2}\,dy = \int_0^1 \frac{g(\frac{1}{y}-1)}{y^2}\,dy$$

~~then, if $U \sim U[0,1]$, $I =$~~ $\boxed{= \int_0^1 h(y)\,dy}$

$$\boxed{h(y) = \frac{g(\frac{1}{y}-1)}{y^2} = \frac{\frac{1}{y}-1}{(1+(\frac{1}{y}-1)^2)^2 y^2}}$$

If $y \sim U[0,1]$, $I = E(h(Y)) \sim \sum_{i=1}^{K} \frac{1}{K}(h(Y_i))$

K large

(Q.7) Note that :-

$$e^{-(-x)^2} = e^{-(x)^2}$$

$\Rightarrow f(-x) = f(x)$

$\Rightarrow f$ symmetric about 0

So, $\int_{-\infty}^{\infty} e^{-x^2} dx$

$= 2\int_0^{\infty} e^{-x^2} dx$

Now we can convert $0-\infty$ to $0-1$

($x \to y$ requires y to be a monotonic f^n of x)

$\Rightarrow y = \frac{1}{x+1}$ | $x=0 \to y=1$, $x \to \infty \Rightarrow y \to 0$

So, $\boxed{I = 2\int_0^{\infty} g(x)\,dx}$ $(g(x) = e^{-x^2})$

$= 2\int_1^0 g\left(\frac{1}{y}-1\right) \cdot \frac{-(dy)}{y^2}$

$= 2\int_0^1 \frac{g\left(\frac{1}{y}-1\right)}{y^2} dy$

$= \boxed{2\int_0^1 h(y)\,dy}$

where $h(y) = \frac{g\left(\frac{1}{y}-1\right)}{y^2} = \boxed{\frac{e^{-\left(\frac{1}{y}-1\right)^2}}{y^2}}$

(Q.8) Note that :-

$$I = \int_0^{\infty}\int_0^{x} e^{-(x+y)} dy\,dx = \int_0^{\infty}\int_0^{\infty} I(y<x) e^{-(x+y)} dy\,dx$$

Now, again the same method :-

$u = \frac{1}{x+1}$, $v = \frac{1}{y+1} \Rightarrow \frac{-du}{u^2} = \frac{-dv}{v^2}$

$x: 0 \to \infty$, $u: 1 \to 0$; $y: 0 \to \infty$, $v: 1 \to 0$

So, $I = \int_1^0\int_1^0 I(u<v) e^{-\left(\frac{1}{u}-1+\frac{1}{v}-1\right)} \left(+\frac{du}{u^2}\right)\left(+\frac{dv}{v^2}\right)$

$= \int_0^1\int_0^1 I(u<v) \frac{e^{-\left(\frac{1}{u}+\frac{1}{v}-2\right)}}{u^2 v^2} dv\,du$

If we assume $u \perp\!\!\!\perp v$

$$\boxed{I = E_{U,V}\left[I(u<v) \frac{e^{-\left(\frac{1}{u}+\frac{1}{v}-2\right)}}{u^2 v^2}\right] \text{ where } U, V \sim U[0,1]}$$

And since $x \perp\!\!\!\perp y$ (no info given about dependence, hence indep)

$f(x) \perp\!\!\!\perp g(y)$ $\forall f, g$

$\Rightarrow \frac{1}{x+1} \perp\!\!\!\perp \frac{1}{y+1}$

$\Rightarrow u \perp\!\!\!\perp v$

$\Rightarrow$ Our assumption is valid.

(Q.10) ~~Cov(U, e^U)~~

Exact Answer :-

$Cov(U, e^U)$

$= E(Ue^U) - E(U)E(e^U)$

$= \int_0^1 x e^x \cdot 1 dx - \frac{1}{2}\int_0^1 e^x \cdot 1 dx$

$= (xe^x)_0^1 - \int_0^1 e^x dx - \frac{1}{2}(e-1)$

$= e - (e-1) - \frac{1}{2}(e-1)$

$= 1 - \frac{1}{2}(e-1)$

$= \frac{2 - e + 1}{2}$

$= \boxed{\frac{3-e}{2}}$

~~$= \frac{3-2.718}{2}$~~

~~$= \frac{0.282}{2}$~~

~~$= 0.141$~~

simulation

$Cov(U, e^U)$

$= \frac{\sum_{i=1}^{k} u_i e^{u_i}}{k} - \frac{\left(\sum_{i=1}^{k} u_i\right)}{k} \frac{\left(\sum_{i=1}^{k} e^{u_i}\right)}{k}$

(Q.12)

Let $U_1, U_2, \ldots \sim U[0,1]$

$N = \min\{n : \sum_{i=1}^{n} U_i > 1\}$

(d) $P(N = \alpha)$

$= P\left(\sum_{i=1}^{\alpha-1} U_i \le 1, \sum_{i=1}^{\alpha} U_i > 1\right)$

$= P\left(\sum_{i=1}^{\alpha-1} U_i \le 1\right) - P\left(\sum_{i=1}^{\alpha-1} U_i \le 1, \sum_{i=1}^{\alpha} U_i \le 1\right)$

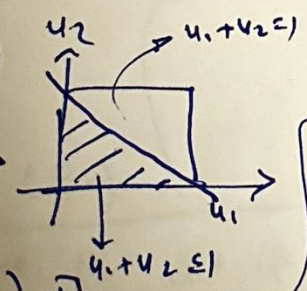
$= P\left(\sum_{i=1}^{\alpha-1} U_i \le 1\right) - P\left(\sum_{i=1}^{\alpha} U_i \le 1\right) P\left(\sum_{i=1}^{\alpha-1} U_i \le 1 \,\middle|\, \sum_{i=1}^{\alpha} U_i \le 1\right)$

$= P\left(\sum_{i=1}^{\alpha-1} U_i \le 1\right) - P\left(\sum_{i=1}^{\alpha} U_i \le 1\right)$

Now, $U \sim [0,1]$

If $Z = U_1 + U_2$, $f_Z(z) = P(Z = z)$

$= P(U_1 + U_2 = z)$



$F_Z(1) = P(Z \le 1) = P(U_1 + U_2 \le 1) = \frac{1}{2}$

for $Z = U_1 + U_2 + U_3$,

$F_Z(1) = P(U_1 + U_2 + U_3 \le 1)$

$= \int_0^1 \int_0^{1-u_1} \int_0^{1-u_1-u_2} du_3\, du_2\, du_1$

$= \int_0^1 \int_0^{1-u_1} (1 - u_1 - u_2)\, du_2\, du_1$

$= \int_0^1 \left[(1-u_1)u_2 - \frac{u_2^2}{2}\right]_0^{1-u_1} du_1$

$= \int_0^1 \frac{(1-u_1)^2}{2}\, du_1$

$= \left[\frac{(1-u_1)^3}{6}\right]_0^1 = \frac{1}{6}$

So, as seen the pattern is:-

$P\left(\sum_{i=1}^{k} U_i \le 1\right) = \frac{1}{k!}$

Hence, $P(N = \alpha) = \frac{1}{(\alpha-1)!} - \frac{1}{\alpha!}$

$= \frac{\alpha - 1}{\alpha!} \quad (\alpha \ge 2)$

$\Rightarrow E(N) = \sum_{\alpha=2}^{\infty} \frac{(\alpha-1)\cancel{\alpha}}{\cancel{\alpha!}(\alpha-1)!}$

$= \sum_{\alpha=2}^{\infty} \frac{1}{(\alpha-2)!}$

$= \frac{1}{0!} + \frac{1}{1!} + \frac{1}{2!} + \ldots$

$= e^1\left(1 + 1 + \frac{1}{2!} + \frac{1}{3!} + \ldots\right)$

$= \boxed{e}$

Chapter 4

(Q.3)

Order these probabilities in the decreasing order of their values so that the most likely one is kept at the top or at the first step of the (uniform) decision making process & so it is highest likely to exit in the first step itself which makes the process the most efficient.

$$H_i = \begin{cases} 0 & \text{w.p. } \frac{99}{100} \\ 1 & \text{w.p. } \frac{1}{100} \end{cases}$$

i.e. i^{th} card to be turned over $= \text{card}_i$

(Q.4) Let N = # hits (RV)

$$\Rightarrow \boxed{N = \sum_{i=1}^{100} H_i \quad \text{where } H_i = 1 \text{ if } i^{th} \text{ card gets a hit (success)}, \; = 0 \text{ otherwise}}$$

$$\Rightarrow E(N) = \sum_{i=1}^{100} E(H_i)$$

$$= \sum_{i=1}^{100} P(H_i = 1)$$

Now, i^{th} card can be placed anywhere in the sequence (of length 100) for $H_i = 1$ i.e. getting a hit, it should be in position i $\Rightarrow$ prob $= 1/100$

$$\Rightarrow \boxed{E(N) = \sum_{i=1}^{100} \frac{1}{100} = 1}$$

Now, to compute variances,

$$\text{var}(N) = E(N^2) - (E(N))^2$$

$$E(N^2) = \sum_{i=1}^{100} E[(H_i)^2] + \sum_{i \neq j} E(H_i H_j)$$

$$= \sum_{i=1}^{100} \frac{1}{100} + \sum_{i \neq j} E(H_i H_j)$$

$$= 1 + \sum_{i \neq j} E(H_i H_j) \quad -(*)$$

Now, $E(X_i X_j)$ a (Assuming $i \neq j$)

$= P$ (card$_i$ in i^{th} card/position & card$_j$ in j^{th} card/position)

$= P$(card$_i$ in i^{th} posn) P(card$_j$ in j^{th} posn / card$_i$ in i^{th} posn)

$= \frac{1}{100} \cdot \frac{1}{99}$ — ①

using ① in ⊛,

$$E(N^2) = 1 + \sum_{i \neq j} \frac{1}{100} \cdot \frac{1}{99}$$

$$= 1 + \frac{100 \cdot 99}{100 \cdot 99}$$

$$= 1 + 1$$

$$= 2$$

$\Rightarrow var(N) = E(N^2) - \cdot$

$= 2 - (1)^2$

$= 2 - 1$

$= \boxed{1}$

$\Rightarrow \boxed{var(N) = 1}$

So, exact answers i.e.

$\boxed{E(N) = 1,\ var(N) = 1}$

match with our estimates

(Q5)(b) By mathematical Induction,

$\boxed{\text{Base case}}$:- $n=1$ = [1]

⇒ The only possible permutation is [1] & that is there ✓

$\boxed{\text{Induction}}$: Assume that the claim holds for $n-1$.

i.e. The permutation obtained for $n-1$ (by the algo) is equally likely to be any of the $(n-1)!$ permutations.

$\boxed{\text{Proof}}$ (refer to ⊛ point at the end) — Let the permt realized for $n-1$ be :-

~~be $(P_1, P_2, \ldots, P_{n-1})$ (w) prob $\frac{1}{(n-1)!}$~~

step 1 (S1) $\underbrace{P_1^\circ, P_2^\circ, \ldots, P_{n-1}^\circ}$ ⇒ $L = P_1^\circ, P_2^\circ, \ldots, P_{n-1}^\circ$

$\boxed{\frac{1}{(n-1)!}}$

prob that we get some specific realization $P_1^\circ, P_2^\circ, \ldots, P_{n-1}^\circ$

(S2) (due to our assumption)

Next, $L' = L + [n] = P_1^\circ, P_2^\circ, \ldots, P_{n-1}^\circ, n$

(S3) Now, in the next step, we randomly pick an index i from $\{1, 2, \ldots, n\}$

$P(\text{index} = i) = \frac{1}{n}$

& then we swap $L[i]$ & $L[n]$ $(=n)$

⇒ The new sequence we get :-

$L'' = P_1^\circ, P_2^\circ, \ldots, P_{i-1}^\circ, n, P_{i+1}^\circ, \ldots, P_{n-1}^\circ, P_i^\circ$

~~$P(L'' \mid L(\text{or } P_1, P_2, \ldots, P_{n-1})) = \boxed{\frac{1}{n}}$~~

~~So, $P(L'', L) = P(L'' \mid L)\, P(L)$~~

~~$= \frac{1}{n} \frac{1}{(n-1)!}$~~

~~$= \frac{1}{n!}$~~ (further, whatever it generates is a valid sequence i.e. it only contains values from 1 to n ∈ Theoretically poss set of length-n seq)

~~So, $P(L'')$. Note that :-~~

~~if we show that $P(L'') = \frac{1}{n!}$, then~~

~~since L'' is generic / not a specific eg, L'' is an output of the algorithm formed using the algorithm, L'' is a placeholder for the outp~~

~~In general,~~

Note that :-

~~L'' is / represents~~ (from ~~step~~ (S1)) $\boxed{=}$ since the L (length $n-1$) sequence we assumed, is variabilized / generic & we just followed the algorithm there after,

L'' is / represents any possible output of the algorithm.

To that end, L'' is a generic variable storing (can store) all possible realizations (w) of the output $= [P_1^\circ, \ldots, P_{i-1}^\circ, n, P_{i+1}^\circ, \ldots, P_i^\circ]$

So, if we show that $P(L'') = \frac{1}{n!}$, that inherently means :-

$P(L'' = (w)) = \frac{1}{n!} \quad \forall w$

⇒ # w that this algorithm (thm) generates to make prob sum to 1 is $\frac{1}{p} = \frac{1}{\frac{1}{n!}} = n!$

⇒ [So, the algorithm generates $n!$ distinct sequences of length n, each equally likely].

[Since there are only $n!$ distinct such seq possible, the algo generates all possible sequences (of length n) with equal likelihood].

Now, $P(L'' = [P_1^\circ, P_2^\circ, \ldots, P_{i-1}^\circ, n, P_{i+1}^\circ, \ldots, P_{n-1}^\circ, P_i^\circ])$

as per the algo step

$\overset{S2 \to S3}{=} P(L' = [P_1^\circ, P_2^\circ, \ldots, P_{n-1}^\circ, n], \text{index} = i)$

$\overset{S1 \to S2}{=} P(L = [P_1^\circ, P_2^\circ, \ldots, P_{n-1}^\circ], \text{index} = i)$

$\overset{\because S1 \perp\!\!\!\perp S3}{=} P(L = [P_1^\circ, P_2^\circ, \ldots, P_{n-1}^\circ]) \cdot P(\text{index} = i)$

$= \frac{1}{(n-1)!} \cdot \frac{1}{n}$ $\to \because$ n possible indices

By induction assumption

$= \frac{1}{n!}$

— QED

(*) Note that in the (proof) part, L, L', L'' are the variables storing the values realized at S1, S2, S3. In general, they are not fixed & can take ~~possible / multiple~~ different values in different iterations.

```
import math
import numpy as np
```

Chapter 3

As per the book's assumption, we assume that we have a black box that gives a random number upon request (which is np.random.rand() in this case)

```
def simulate_integral_value(k, h):
    ans = 0
    for _ in range(k):
        y = np.random.rand() # U[0, 1]
        ans += h(y)
    ans = ans/k
    return ans
```

Q6

```
# k = int(1e6)
# def h(y):
#     nmr = 1/y - 1
#     dnm = ((1+(1/y-1)**2)*y)**2
#     return nmr/dnm
# ans = 0
# for i in range(k):
#     y = np.random.rand()
#     ans += h(y)
# ans = ans/k
# print(ans)

k = int(1e6)
def h(y):
    nmr = 1/y - 1
    dnm = ((1+(1/y-1)**2)*y)**2
    return nmr/dnm
ans = simulate_integral_value(k, h)
print(ans)

0.49995430528505924
```

Matches with the exact answer

Q7

```
k = int(1e6)
def h(y):
    nmr = math.exp(-(1/y - 1)**2)
    dnm = y**2
    return nmr/dnm
```

```
ans = 2*simulate_integral_value(k, h)
print(ans)
```

```
1.771241916712693
```

Matches with the exact answer

Q9

```
def simulate_integral_value_2d(k, h):
    ans = 0
    for _ in range(k):
        u = np.random.rand() # U[0, 1]
        v = np.random.rand() # U[0, 1]
        ans += h(u, v)
    ans = ans/k
    return ans

k = int(1e6)
def h(u, v):
    if(u>=v):
        return 0
    return math.exp(-(1/u+1/v-2))/((u*v)**2)
ans = simulate_integral_value_2d(k, h)
print(ans)
```

```
0.49952046543913003
```

Matches with the exact answer

Q10

```
k = int(1e6)
t1, t2, t3 = 0, 0, 0
for _ in range(k):
    u = np.random.rand()
    t1 += u*math.exp(u)
    t2 += u
    t3 += math.exp(u)
t1, t2, t3 = t1/k, t2/k, t3/k
ans = t1-t2*t3
print(ans)
```

```
0.1407680176628976
```

Matches with the exact answer

Q12 (a, b, c)

```
kvec = [100, 1000, 10000]
for k in kvec:
```

```
    for _ in range(k):
        cnt = 0
        val = 0
        while(val<=1):
            u = np.random.rand()
            cnt += 1
            val += u
        ans += cnt
    ans = ans/k
    print(k, ans)
```

```
100 2.661407680176629
1000 2.6886614076801765
10000 2.723468866140768
```

Q14

```
x = np.zeros(15)
x[1], x[2] = 23, 66
for i in range(3, 15):
    x[i] = (3*x[i-1]+5*x[i-2])%100
u = x/100
random_number_sequence = u[1:]
print(u[1:])
```

```
[0.23 0.66 0.13 0.69 0.72 0.61 0.43 0.34 0.17 0.21 0.48 0.49 0.87
0.06]
```

Chapter 4

Q1

```
# Program to generate n values from the pmf with p1 = 1/3 and p2 = 2/3
p1 = 1/3
def simulate_pmf12(n):
    output = []
    v = np.random.rand(n)
    for val in v:
        if(val<=p1):
            output.append(1)
        else:
            output.append(2)
    output = np.array(output)
    return output

for n in [100, 1000, 10000]:
    output = simulate_pmf12(n)
    assert(len(output) == n)
    prop = len(output[output == 1])/len(output)
    print(n, prop)
```

```
100 0.33
1000 0.324
10000 0.326
```

Q2

```
def simulate_pmf_n(pvec) -> float:
    '''
    Args:
        pvec: vector of probabilities defining the probability mass
function (pmf)
    Returns:
        value of the Random Variable following the given pmf
    '''
    n = len(pvec)
    v = np.random.rand()
    # The cdf value (upto index i; i ranging from 1 to n)
    F = p1
    ans = -1
    for i in range(1, n+1):
        if(v<=F):
            ans = i
            break
        if(i==n):
            raise Exception("index of iteration should not reach n!")
        F += pvec[i]
    assert(ans != -1)
    return ans
```

Q3

```
u = np.random.rand()
if(u<=0.35):
    ans = 3
elif(u<=0.65):
    ans = 1
elif(u<=0.85):
    ans = 2
else: # u will surely be <= 1
    ans = 4
print(ans)
```

```
3
```

Q4

```
n = 100
v = []

num_trials = 50000
```

```
for _ in range(num_trials):
    # seq will represent the sequence of shuffled cards

    # Creating a base sequence
    seq = np.array([i for i in range(n+1)])

    # Shuffling the sequence
    for k in range(n, 1, -1):
        u = np.random.rand()
        index = int(k*u) + 1

        # swapping seq[index] and seq[k]
        temp = seq[k]
        seq[k] = seq[index]
        seq[index] = temp

    # counting the number of hits
    cnt = 0
    for i in range(1, len(seq)):
        if(seq[i] == i):
            cnt += 1
    v.append(cnt)

expectation = np.mean(v)
variance = np.var(v)
print(expectation, variance)
```

```
1.00454 1.0075193884000002
```

Empirical estimates

- Expectation: 1
- Variance: 1

Q5

```
def get_random_permutation_b(n):
    output = [[1]]
    print(output[0])
    for i in range(2, n+1):
        # previous sequence
        seq = output[-1]
        # perm is of length i
        perm = seq + [i]
        u = np.random.rand()
        index = int(i*u) + 1

        # swapping the 2 numbers
        temp = perm[index-1]
        perm[index-1] = perm[i-1]
        perm[i-1] = temp
```

```
        print(perm)
        output.append(perm)
    return output

output = get_random_permutation_b(6)
```

```
[1]
[1, 2]
[3, 2, 1]
[3, 4, 1, 2]
[3, 5, 1, 2, 4]
[3, 5, 1, 2, 4, 6]
```

Q6

Method 1

```
# u = np.random.rand()
u = random_number_sequence[0]
prob = 0.8
len_seq = 25
output = np.zeros(len_seq)
for i in range(len_seq):
    if(u<=prob):
        val = 1
    else:
        val = 0
    output[i] = val
    # updating u for the next sampling
    if(u<=prob):
        u = u/prob
    else:
        u = (u-prob)/(1-prob)
output
```

```
array([1., 1., 1., 1., 1., 1., 0., 1., 1., 1., 1., 0., 1., 0., 1., 1.,
0.,
       1., 1., 1., 1., 1., 1., 1., 1.])
```

```
len(output[output==1])
```

```
21
```

just 1 random number was needed!

Method 2

```
prob = 0.80
i = 0
output = []
```

```
while(True):
    u = random_number_sequence[i]
    x = int(math.log(u)/math.log(prob)) + 1
    for _ in range(x-1):
        output.append(1)
        if(len(output) == 25):
            break
    if(len(output) == 25):
        break
    output.append(0)
    if(len(output) == 25):
        break
    i += 1
output
```

```
[1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 0,
1, 1]
```

Since there are 6 sequences of 1 (or starting with 1), 6 random numbers were needed!

Q7

```
num_trials = int(2e5)
ans = 0
for _ in range(num_trials):
    # cnt represents the number of times we throw the pair of dice;
    #   number of throws required to get all possible sums atleast
once
    cnt = 0
    # s stores only the distinct values; so once s has all the
elements from 2 to 12
    #   (11 possible values), we are guaranteed to have each sum
atleast once
    #   Note that no other value (of the sum) is possible;
    #   so len(s) = 11 <=> all possible sums (from 2 to 12) occured
atleast once
    s = set([])
    while(len(s) != 11):
    # while(len(s) != 6):
        # simulate throwing of 2 dice
        cnt += 1
        # get the value
        u1 = np.random.rand()
        v1 = int(6*u1) + 1
        u2 = np.random.rand()
        v2 = int(6*u2) + 1
        # v2 = 0
        s.add(v1+v2)
    ans += cnt
```

```
ans = ans/num_trials
print(ans)
```

```
61.18592
```

The estimate (expected number of dice rolls) seems to be 61.2 => upper/ceilinged to 62